Worksheet — 2.1 Elements of Computational Thinking — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Task 1 — Key-term match 1 → C, 2 → E, 3 → D, 4 → A, 5 → B.

Task 2 — Inputs / outputs / preconditions - (a) Inputs (any three): student ID; chosen room (room number); chosen date/time slot; (optionally) duration / number of people. - (b) Outputs (any two): the list of available rooms displayed; a confirmation message / booking reference; or a rejection message if unavailable. - (c) Precondition (any one): the student ID must be valid/registered before a booking is accepted; **or** the chosen room and slot must exist; **or** the chosen slot must currently be free. (Note: validating the ID and checking availability are *processes*, not inputs/outputs.)

Task 3 — Abstraction (keep/remove) - KEEP: room number/ID, maximum capacity, which slots are already booked — these are needed to decide and store a booking. - REMOVE: carpet colour, make of chairs — irrelevant to whether a room can be booked. They are irrelevant *to this problem* because the booking decision depends only on identity, capacity and availability, never on décor.

Task 4 — Decomposition (model order) 1. Get/validate the student ID. 2. Display the list of available rooms. 3. Get the student's chosen room and time slot. 4. Check the chosen slot is still free (availability check). 5. Store the booking in the database. 6. Print/display the confirmation (or rejection). Dependency example: step 5 (store) depends on the output of step 4 (the availability check must pass first); step 4 depends on the chosen room/slot from step 3.

Task 5 — Decision points (examples) - DP1: `IF studentID is valid` → TRUE: continue to room selection; FALSE: show "unknown ID / please register" and stop. - DP2: `IF chosenSlot is free` (e.g. `IF slotStatus = "free"`) → TRUE: store the booking and confirm; FALSE: reject and ask the student to choose another slot. - (Other valid conditions: `IF seatsRequested <= roomCapacity`, `IF bookingTime >= openingTime AND bookingTime <= closingTime`.)

Task 6 — Concurrency - (a) Independent (no data dependency): e.g. loading the available-rooms list for the screen **while** the system logs the previous request; refreshing one room's availability while another student's confirmation email is sent; rendering the UI while a background database query runs. - (b) Sequential (must stay ordered): you must **check availability before storing the booking**, and **store the booking before printing the confirmation** — each needs the previous step's result. - (c) Benefit: faster/more responsive — the interface stays usable while a slow query runs, and multiple students can be served at once on multi-core hardware. Trade-off: harder to design and debug; risk of a **race condition** if two students book the same slot at the same instant (needs synchronisation/locking), and concurrency adds overhead so it is not automatically faster.

Task 7 — Abstraction vs decomposition 1. A 2. D 3. A 4. D 5. A 6. D

Challenge box — caching Caching stores the "rooms free" list in fast memory so repeated requests are served without re-querying the database each time — this is faster, reduces database load/traffic and improves responsiveness when many students ask at once. Trade-off: it uses extra storage, and the cached list can become **stale** when a booking or cancellation changes availability but the cache still

shows the old state — a student could be shown a room that has just been taken. Handle this by **invalidating/updating the cache** whenever a booking or cancellation occurs, or by setting a short expiry time so the list is refreshed regularly.